

Plan de correction — Collisions de créneaux 2025-2026

Fichier de référence décrivant l'analyse de la cause racine, les correctifs apportés, et la marche à suivre pour régénérer les contraintes et rejouer les tests. Toutes les commandes sont à exécuter depuis la racine du dépôt

LabScheduling/ .

1. Problème constaté

Sur les emplois du temps 2025-2026, **99 collisions sur 131 (75,6 %)** ont été détectées : des TP étaient planifiés sur des créneaux où les étudiants concernés avaient déjà **cours magistral** d'après les grilles « Horarios » réelles fournies par Daniel.

Cause racine

Le fichier `data_clean/optimization/student_busy.csv` (indisponibilités « cours » des étudiants) et `master_schedule.csv` **n'étaient pas synchronisés avec les vraies grilles Horarios 2025-2026**. Les créneaux occupés y étaient **décalés** (mauvais bloc horaire / mauvais jour) par rapport à la réalité.

Comme le solveur CP-SAT s'appuie sur `student_busy` pour interdire les créneaux « cours », un décalage dans cette source se traduit **mécaniquement** par des TP placés en plein cours magistral. Le solveur faisait donc « bien son travail » sur des données fausses : la correction devait porter sur la **source de vérité**, pas sur le solveur.

2. Solutions apportées (par tâche)

#	Objectif	Livrable principal
1	Régénérer <code>student_busy.csv</code> depuis les vraies grilles Horarios	<code>re-build_student_constraints.py</code> , <code>horarios_grid.py</code>
2	Vue « Horarios » dans l'export Excel (cours + TP + collisions)	<code>excel_generator_core.py</code> , <code>excel_export.py</code>
3	Exclure des semaines de TP par matière + globalement	<code>pipeline.py</code> , <code>app.py</code>
4	Détection de collisions dans « Éditer le plan »	<code>manual_edit.py</code> , <code>app.py</code>
5	Validation pré-export (bloquer sur collisions critiques)	<code>pre_export_validation.py</code> , <code>pipeline.py</code>
6	Documentation + tests unitaires + test pipeline complet	ce fichier, <code>tests/</code>

TASK 1 — Reconstruire `student_busy` depuis la source de vérité

- `horarios_grid.py` : parseur partagé des grilles « Horarios ». Construit une grille d'occupation (`titulación, curso`) → {(jour, bloc): cours}. Fonctions clés : `load_occupancy_grid`, `parse_horarios_sheet`, `normalize_titulacion`, `busy_slots_for`.
- `rebuild_student_constraints.py` : dérive `student_busy.csv` (et `titulacion_busy.csv`) à partir de la grille réelle, avec un rapport de couverture. **Couverture : 91,8 % (436/475 étudiants, 5609 entrées).**
- `pipeline.py` : quand les grilles réelles sont disponibles, le pipeline remplace la dérivation par la source de vérité (flag `LAB_USE_CORRECTED_HORARIOS`, activé par défaut).

TASK 2 — Vue « Horarios » dans l'export Excel

Nouvelle feuille par semestre : jours en colonnes, blocs horaires en lignes. Les **cours magistraux** apparaissent en gris, les **TP planifiés** en vert, et toute **collision** (TP sur un créneau de cours) est surlignée en rouge — pour un contrôle visuel immédiat.

TASK 3 — Semaines exclues (par matière + global)

- Option par matière « Semaines exclues (pas de TP) » dans l'onglet configuration, plus une option globale « Semaines globalement exclues ».
- Le solveur retire ces semaines du domaine des variables « semaine » (avec relâchement gracieux si le domaine devient vide).

- Persisté dans `user_config.json` (`global.excluded_weeks_all` et `subjects[...].excluded_weeks`).
- **Cas d'usage** : Chimie 1re année / S1 → exclure les semaines 7, 8, 11, 12.

TASK 4 — Détection de collisions dans « Éditer le plan »

- `manual_edit.py` : `validate_edit_collision(...)` retourne un `EditCollisionReport` catégorisant les conflits **étudiant / professeur / salle / matière**, et propose des créneaux libres alternatifs.
- Option « **Forcer le déplacement** » : l'utilisateur peut passer outre ; les conflits sont alors convertis en avertissements `[FORCÉ]` et le déplacement est appliqué.

TASK 5 — Validation pré-export

- `pre_export_validation.py` : `validate_complete_schedule(...)` renvoie un `ValidationReport` (listes C1 matière, C4 salle, professeur, étudiant, signal C7, + score de qualité /100).
- Intégré dans `pipeline.run_pipeline()` **avant** la génération des exports Excel. Écrit `reports/pre_export_validation.json`.
- **Comportement par défaut : non bloquant** (avertit fortement + rapport JSON), pour ne pas casser une démo sur un planning hérité contenant des collisions réelles. Pour **bloquer** réellement l'export en cas de collision critique, définir `LAB_BLOCK_EXPORT_ON_COLLISION=1`.

TASK 7 — Vérification formelle CP-SAT (`cpsat_verifier.py`)

Une **couche de preuve formelle** vient compléter la validation heuristique ci-dessus. Là où `pre_export_validation.py` fait une détection rapide par paires, `cpsat_verifier.py` encode le planning dans un **modèle CP-SAT** (OR-Tools) et **prouve** exhaustivement sa cohérence.

- **Principe** : chaque séance est figée à son créneau réel (semaine, jour, bloc, salle, professeur). Toutes les contraintes dures sont posées sous forme de variables de violation réifiées, dont on **minimise la somme**.
- Coût minimal = **0** ⇒ le planning est formellement valide (preuve).
- Coût > **0** ⇒ chaque violation est identifiée précisément (nature, séances, créneau) — pas d' `INFEASIBLE` opaque.
- **Contraintes vérifiées** : C1 (matière), professeur (double-réservation interne + indisponibilité externe `professor_busy`), salle physique (double-booking + réservation externe), semaines exclues (`excluded_weeks_for`), collisions étudiant (double-TP + TP vs cours via `group_composition` / `student_busy`), et préférence horaire par année (signal **non bloquant**).
- **API** : `verify_schedule(schedule_df, ...)` → `VerificationResult` avec `is_feasible`, `violations`, `n_violations_by_type`, `solver_status`, `wall_time`, `format_text()`, `to_dict()`.
- **Intégration** : `pipeline.run_pipeline()` appelle `cpsat_verifier.run_cpsat_verification_gate(...)` **après** la porte heuristique. Écrit `reports/cpsat_verification.json`. **Non bloquant par défaut** (comme la porte heuristique) ; passer

LAB_BLOCK_EXPORT_ON_CPSAT_FAIL=1 pour bloquer l'export sur violation dure,
ou LAB_CPSAT_VERIFY=0 pour désactiver la couche.

- **CLI :**

```
bash
```

```
python cpsat_verifier.py --schedule outputs/optimization/optimized_schedule_v5.csv
```

```
python cpsat_verifier.py --schedule <csv> --semester 1 # restreindre à un semestre
```

- **Différence avec pre_export_validation.py** : la validation heuristique donne un feedback immédiat (rapide, par paires) ; le vérificateur CP-SAT fournit une garantie formelle exhaustive (modèle de contraintes) — les deux sont cohérents (mêmes types de collisions, mêmes chemins CSV, même convention de créneaux jours 0-4 / blocs 1-6).

3. Régénérer les contraintes (guide)

```
# 1. Régénérer student_busy.csv depuis les grilles Horarios réelles
python rebuild_student_constraints.py

# (options utiles)
python rebuild_student_constraints.py --help
# Écrit :
#   data_clean/optimization/student_busy.csv
#   data_clean/optimization/titulacion_busy.csv
#   reports/rebuild_student_constraints_report.txt (rapport de couverture)

# 2. Rejouer le pipeline complet avec les contraintes corrigées
python pipeline.py
# Le flag LAB_USE_CORRECTED_HORARIOS=1 (défaut) fait utiliser la source réelle.

# 3. Valider un planning existant à la main (facultatif)
python pre_export_validation.py outputs/optimization/optimized_schedule_v5.csv
# Sortie : rapport lisible + code retour 1 s'il y a des collisions critiques.
```

Variables d'environnement

Variable	Défaut	Effet
LAB_USE_CORRECTED_HORARIOS	1	Utiliser les grilles Horarios réelles pour <code>student_busy</code> . Mettre à 0 pour l'ancien comportement.
LAB_BLOCK_EXPORT_ON_COLLISION	0	Bloquer l'export Excel si des collisions critiques sont détectées (porte heuristique).
LAB_CPSAT_VERIFY	1	Exécuter la vérification formelle CP-SAT après la porte heuristique. Mettre à 0 pour la désactiver.
LAB_BLOCK_EXPORT_ON_CPSAT_FAIL	0	Bloquer l'export Excel si le vérificateur CP-SAT prouve des violations dures.

4. Tests

Tests unitaires ajoutés (tests/)

- `test_horarios_grid.py` — normalisation titulación, libellés de créneaux, parsing d'une mini-grille, `busy_slots_for`.
- `test_rebuild_student_constraints.py` — conversion année→curso, dérivation `student_busy` (couverture, semaines manquantes).
- `test_pre_export_validation.py` — détection C1 / C4 / professeur / C7, score, sérialisation du rapport, porte `run_pre_export_gate` (bloquant / non bloquant).
- `test_excluded_weeks.py` — `excluded_weeks_for` (global, par matière, union, valeurs malformées).
- `test_cpsat_verifier.py` — vérification formelle CP-SAT : planning valide, C1, professeur (interne + externe), salle, semaine exclue, collisions étudiant, signal préférence, rapport, porte `run_cpsat_verification_gate`.

Lancer les tests

```
# Suite complète (doit rester au vert)
python -m pytest -q

# Uniquement les tests des correctifs
python -m pytest tests/test_horarios_grid.py \
                 tests/test_rebuild_student_constraints.py \
                 tests/test_pre_export_validation.py \
                 tests/test_excluded_weeks.py -q
```

État actuel : 309 tests au vert (267 de référence + 42 ajoutés). Aucune régression.

5. Vérification de la correction (avant / après)

1. **Avant** : sur les données 2025-2026 non corrigées → 99/131 collisions (75,6 %) causées par le décalage de `student_busy`.
2. **Correctif** : `rebuild_student_constraints.py` reconstruit `student_busy` depuis les grilles Horarios réelles (91,8 % de couverture).
3. **Après** : rejouer `python pipeline.py`, puis contrôler `reports/pre_export_validation.json` et la feuille « Horarios » de l'export Excel — les TP ne doivent plus tomber sur des créneaux de cours magistral.

Note : le planning `optimized_schedule_v5.csv` livré dans le dépôt a été généré **avant** la correction ; il contient encore des collisions professeur héritées. Elles servent de démonstration que la validation pré-export les détecte bien. Après un `python pipeline.py` avec les contraintes corrigées, ce nombre doit chuter fortement.

6. Fichiers modifiés / ajoutés

Ajoutés

- `horarios_grid.py`
- `rebuild_student_constraints.py`
- `pre_export_validation.py`
- `cpsat_verifier.py` — vérification formelle CP-SAT.
- `tests/test_horarios_grid.py`
- `tests/test_rebuild_student_constraints.py`
- `tests/test_pre_export_validation.py`
- `tests/test_excluded_weeks.py`
- `tests/test_cpsat_verifier.py`
- `CORRECTION_PLAN.md` (ce fichier)

Modifiés

- `pipeline.py` — grilles réelles, semaines exclues, porte pré-export, porte de vérification formelle CP-SAT.
- `app.py` — UI semaines exclues (par matière + global), rapport de collisions dans « Éditer le plan ».
- `manual_edit.py` — `validate_edit_collision`, option « Forcer ».
- `excel_generator_core.py`, `excel_export.py` — vue « Horarios ».